Table of Contents

Introduction 2

Data Access Layer (DAL) 3

Modify Controller 5

Register Service 8

Service Lifetime 11

Demo: Shop database, expanded CRUD

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT.

Introduction

This demo introduces important design patterns and concepts of ASP.NET MVC to enhance application structure and maintainability. You will learn:

Data Access Layer (DAL)

- **Repository Pattern**: Implemented to separate data access logic from business logic, improving maintainability and testability.
- **Folder Structure**: A new folder `DAL` is created to contain `DBInit.cs`, `ItemDbContext.cs`, `ItemRepository.cs`, and `IItemRepository.cs`.
- **Purpose**: The repository pattern provides a unified interface for data access, enabling easier switching between different data storage implementations.

Modify Controller

- **Controller Updates**: `ItemController` is modified to use `ItemRepository` methods for data access, separating concerns and streamlining controller logic.

Register Service

- **Service Registration**: The `ItemRepository` service is registered in the dependency injection container, allowing for dependency injection throughout the application.
- **Outcome**: The application now separates data access and business logic, demonstrating a cleaner and more modular codebase.

Service Lifetime

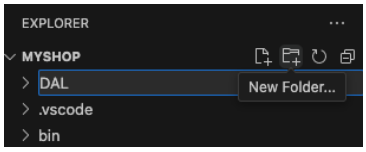
- **Scoped**: Creates a new instance per HTTP request, shared within the request scope.
- **Transient**: Generates a new instance each time it is requested, suitable for stateless operations.
- **Singleton**: Maintains a single instance for the entire application lifetime, ideal for shared data or state management.

The demo covers the implementation of the Repository Pattern for clean data access, updates to controllers for improved logic separation, and the registration and management of services with varying lifetimes. Happy coding!

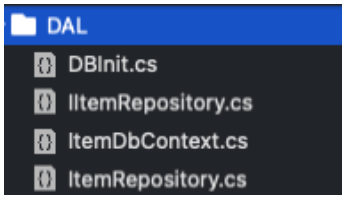
Data Access Layer (DAL)

In this demo, we introduce a design pattern, called Repository Pattern, which serves as the data access layer of web applications.
We can see that as the web application grows, the code becomes more and more complicated.
The repository pattern separates the **data access logic** from the **business logic** (Controllers).
This decoupling provides a unified interface for data access, so that it allows switching between data storage implementations without impacting the rest of the application.
This also makes the codes more maintainable and easier to test with unit testing.

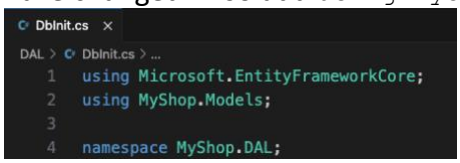
First we create a new folder named as DAL under the project folder.

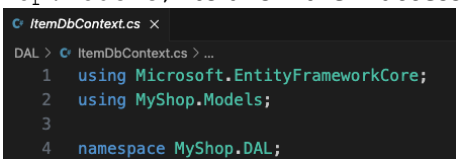


We then move DBInit.cs and ItemDbContext.cs into DAL.
Then we create two classes named `ItemRepository.cs` and `IItemRepository.cs` under DAL (yes, there are two "I"s. The first I stands for interface).
The DAL folder has the following four files:

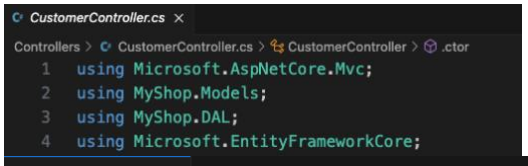


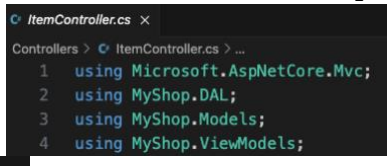
Remember to modify the name space of DBInit.cs and ItemDbContext.cs, since their folders have changed. Also add `using MyShop.Models;` to allow them access the data models.

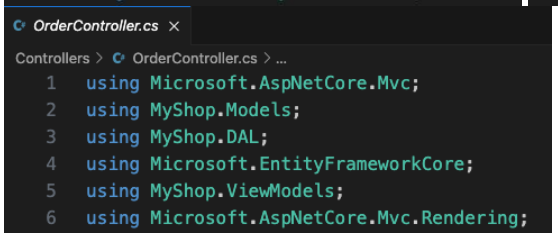




Also remember to add directive that allows the controllers access `MyShop.DAL`:







Modify the IItemRepository to this:

IItemRepository.cs

```
selection
 1  using MyShop.Models;
 2
 3  namespace MyShop.DAL;
 4
 5  public interface IItemRepository
 6  {
 7      Task<IEnumerable<Item>> GetAll();
 8      Task<Item?> GetItemById(int id);
 9      Task Create(Item item);
10      Task Update(Item item);
11      Task<bool> Delete(int id);
12  }
```

Modify the ItemRepository to this (note that we use async database access):

ItemRepository.cs

```
emRepository › Update(Item item)
 1  using Microsoft.EntityFrameworkCore;
 2  using MyShop.Models;
 3
 4  namespace MyShop.DAL;
 5
 6  public class ItemRepository : IItemRepository
 7  {
 8      private readonly ItemDbContext _db;
 9
10      public ItemRepository(ItemDbContext db)
11      {
12          _db = db;
13      }
14
15      public async Task<IEnumerable<Item>> GetAll()
16      {
17          return await _db.Items.ToListAsync();
18      }
19
20      public async Task<Item?> GetItemById(int id)
21      {
22          return await _db.Items.FindAsync(id);
23      }
24
25      public async Task Create(Item item)
26      {
27          _db.Items.Add(item);
28          await _db.SaveChangesAsync();
29      }
30
31      public async Task Update(Item item)
32      {
33          _db.Items.Update(item);
34          await _db.SaveChangesAsync();
35      }
36
37      public async Task<bool> Delete(int id)
38      {
39          var item = await _db.Items.FindAsync(id);
40          if (item == null)
41          {
42              return false;
43          }
44
45          _db.Items.Remove(item);
46          await _db.SaveChangesAsync();
47          return true;
48      }
49  }
```

Modify Controller

Then modify the Controllers/ItemController.cs to this:



```
ItemController.cs
Controllers > ItemController.cs > ...
1   using Microsoft.AspNetCore.Mvc;
2   using MyShop.DAL;
3   using MyShop.Models;
4   using MyShop.ViewModels;
5
6   namespace MyShop.Controllers;
7
    1 reference
8   public class ItemController : Controller
9   {
        9 references
10      private readonly IItemRepository _itemRepository;
11
        0 references
12      public ItemController(IItemRepository itemRepository)
13      {
14          _itemRepository = itemRepository;
15      }
16
        3 references
17      public async Task<IActionResult> Table()
18      {
19          var items = await _itemRepository.GetAll();
20          var itemsViewModel = new ItemsViewModel(items, "Table");
21          return View(itemsViewModel);
22      }
23
        0 references
24      public async Task<IActionResult> Grid()
25      {
26          var items = await _itemRepository.GetAll();
27          var itemsViewModel = new ItemsViewModel(items, "Grid");
28          return View(itemsViewModel);
29      }
30
        0 references
31      public async Task<IActionResult> Details(int id)
32      {
33          var item = await _itemRepository.GetItemById(id);
34          if (item == null)
35              return BadRequest("Item not found.");
36          return View(item);
37      }
38
```

`using Microsoft.EntityFrameworkCore;` is no longer needed because the direct database access is not done in the Controllers but in the repository.
All direct database access via _itemDbContext is now removed. Instead, ItemController.cs uses the methods defined in ItemRepository to access the database.
Line 10-Line15: The constructor uses dependency injection to receive an instance of **`IItemRepository`**.

- `ItemController(IItemRepository itemRepository)`: A constructor that takes an IItemRepository parameter.
- `IItemRepository itemRepository`: The parameter used to inject an instance of the `IItemRepository` interface into the controller.
- `_itemRepository = itemRepository;`: Assigns the injected `IItemRepository` instance to the `_itemRepository` field, allowing the controller to use it for data operations.

ItemControllers.cs continued:

```
39          [HttpGet]
40          public IActionResult Create()
41          {
42              return View();
43          }
44
45          [HttpPost]
46          public async Task<IActionResult> Create(Item item)
47          {
48              if (ModelState.IsValid)
49              {
50                  await _itemRepository.Create(item);
51                  return RedirectToAction(nameof(Table));
52              }
53
54              return View(item);
55          }
56
57          [HttpGet]
58          public async Task<IActionResult> Update(int id)
59          {
60              var item = await _itemRepository.GetItemById(id);
61              if (item == null)
62              {
63                  return NotFound();
64              }
65              return View(item);
66          }
67
68          [HttpPost]
69          public async Task<IActionResult> Update(Item item)
70          {
71              if (ModelState.IsValid)
72              {
73                  await _itemRepository.Update(item);
74                  return RedirectToAction(nameof(Table));
75              }
76
77              return View(item);
78          }
79
80          [HttpGet]
81          public async Task<IActionResult> Delete(int id)
82          {
83              var item = await _itemRepository.GetItemById(id);
84              if (item == null)
85              {
86                  return NotFound();
87              }
88              return View(item);
89          }
90
91          [HttpPost]
92          public async Task<IActionResult> DeleteConfirmed(int id)
93          {
94              await _itemRepository.Delete(id);
95              return RedirectToAction(nameof(Table));
96          }
97      }
```

In addition, we need to modify the `CustomerController.cs` and the `OrderController.cs` to add `using MyShop.DAL.` `using Microsoft.EntityFrameworkCore;` is still needed here because their repository patterns are not created yet and they still rely on direct access to `ItemDbContext.cs`.

CustomerController.cs U

3-EntityFramework > Demo-ShopDatabase-6-Repository > MySh

```
1  using Microsoft.AspNetCore.Mvc;
2  using MyShop.Models;
3  using Microsoft.EntityFrameworkCore;
4  using MyShop.DAL;
```

OrderController.cs U

3-EntityFramework > Demo-ShopDatabase-6-Repository > MyShop > Co

```
1  using Microsoft.AspNetCore.Mvc;
2  using MyShop.Models;
3  using Microsoft.EntityFrameworkCore;
4  using MyShop.ViewModels;
5  using Microsoft.AspNetCore.Mvc.Rendering;
6  using MyShop.DAL;
```

Register Service

We need to register a new service in Program.cs (Line 13)

```
Program.cs
Program.cs
1  using Microsoft.EntityFrameworkCore;
2  using MyShop.DAL;
3
4  var builder = WebApplication.CreateBuilder(args);
5
6  builder.Services.AddControllersWithViews();
7
8  builder.Services.AddDbContext<ItemDbContext>(options => {
9      options.UseSqlite(
10         builder.Configuration["ConnectionStrings:ItemDbContextConnection"]);
11 });
12
13 builder.Services.AddScoped<IItemRepository, ItemRepository>();
14
15 var app = builder.Build();
16
17 if (app.Environment.IsDevelopment())
18 {
19     app.UseDeveloperExceptionPage();
20     DBInit.Seed(app);
21 }
22
23 app.UseStaticFiles();
24
25 app.MapDefaultControllerRoute();
26
27 // app.MapControllerRoute(
28 //     name: "default",
29 //     pattern: "{controller=Home}/{action=Index}/{id?}");
30
31 app.Run();
```

Now if we build the project, we will see errors in Migrations.

```
[USER]:MyShop % dotnet build
Restore complete (0,4s)
  MyShop net10.0 failed with 3 error(s) (0,3s)
    /Users/bazho0342/Code/baifan-ai-projects/ITPE3200-26H/3-EntityFramework/Demo-ShopDatabase-6-Repository/MyShop/Migrations/20260830200656_InitDb.Designer.cs(12,23): error CS0246: The type or namespace name 'ItemDbContext' could not be found (are you missing a using directive or an assembly reference?)
    /Users/bazho0342/Code/baifan-ai-projects/ITPE3200-26H/3-EntityFramework/Demo-ShopDatabase-6-Repository/MyShop/Migrations/ItemDbContextModelSnapshot.cs(11,23): error CS0246: The type or namespace name 'ItemDbContext' could not be found (are you missing a using directive or an assembly reference?)
    /Users/bazho0342/Code/baifan-ai-projects/ITPE3200-26H/3-EntityFramework/Demo-ShopDatabase-6-Repository/MyShop/Migrations/20260830201058_ShopDbExpanded.Designer.cs(12,23): error CS0246: The type or namespace name 'ItemDbContext' could not be found (are you missing a using directive or an assembly reference?)

Build failed with 3 error(s) in 0,8s
```

Since we are using DBSeeding, and no longer need migrations, we can choose to back up and remove the Migrations folder.

Handling Migration Build Errors

When you build the project after moving ItemDbContext.cs into the DAL folder, you will encounter compilation errors inside the Migrations folder.

Why New Migrations Are Not Needed

Introducing the Repository Pattern is a **C# architecture change**, not a **database schema change**. Because the underlying tables, columns, and entity relationships (Items, Customers, Orders) have not changed, there is no need to run dotnet ef migrations add.

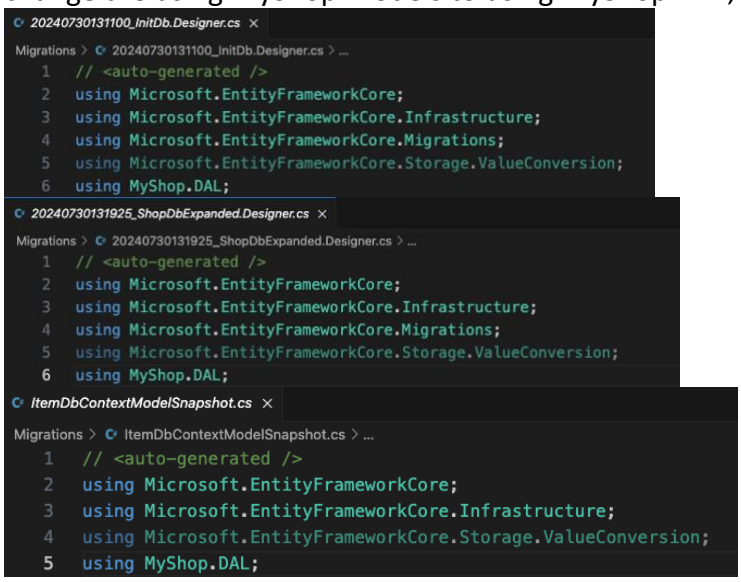
Why We Delete the Migrations Folder in This Lab
The build errors occur because moving ItemDbContext.cs changed its namespace from MyShop.Models to MyShop.DAL. The auto-generated files in the Migrations folder still attempt to reference ItemDbContext under its old MyShop.Models namespace.
Because our development workflow uses DBInit (EnsureDeleted() / EnsureCreated()), EF Core migrations are currently bypassed. Deleting the entire Migrations folder is the simplest way to remove the outdated namespace references and keep the project build green.

Recommended Practice for Architectural Change in Real-World / Production Projects

In a real-world production environment **where live customer data depends on migration history**, developers **never delete the `Migrations` folder**. Instead, they handle namespace refactoring in one of two ways:

- **Namespace Refactoring (Standard Practice):** Update the using directives and namespace declarations inside the existing migration files and snapshot file (ItemDbContextModelSnapshot.cs) to match the new MyShop.DAL namespace.
- **Fresh Initial Migration (Early-Stage Projects):** If the application has not yet been deployed to production, delete the old migration files, drop the local database, and generate a clean baseline migration targeting the new namespace using dotnet ef migrations add InitialCreate.

For the case that we need the migrations, and changed the folder of some files, we can change the using MyShop.Models to using MyShop.DAL; in the Migrations files:



After that, run `dotnet build` to verify that the issues are resolved. Then use the following command to apply the migrations:

```
dotnet ef database update
```

If you have DBSeeding at the same time, this will generate an error SQLite Error 1: 'table "[table name]" already exists'.

Modifying migration files directly is generally not considered best practice due to potential risks and complexities involved. However, there are specific scenarios where changes might be necessary. Here are some considerations and best practices:

- **Avoid Direct Modification:**
  - Reason: Directly modifying migration files can lead to inconsistencies, especially if the migration files have been applied to a production database. Changes can lead to errors or conflicts when updating or rolling back migrations.
  - Recommendation: Migrations should primarily be managed through the Entity Framework Core (EF Core) tools and commands. Use these tools to add, remove, or update migrations rather than editing them manually.
- **Namespace Changes:**
  - Scenario: Moving `ItemDbContext` to a different namespace can necessitate changes in the migration files to reflect the new location of the DbContext class.
  - Recommendation: When making such changes, carefully update the `using` directives in migration files to point to the correct namespace. Ensure you rebuild and test your project to validate that the changes do not introduce errors.
- **Recreating Migrations:**
  - Scenario: If you find that you need to make significant changes to your data model or the initial setup of your migrations is problematic.
  - Recommendation: Consider deleting existing migrations and recreating them if you're in a development environment. This can be done by:
    - Deleting the existing migrations in the `Migrations` folder.
    - Using `dotnet ef migrations add InitialCreate` to create a new initial migration.
    - Using `dotnet ef database update` to apply the new migration to your database.
- **Handling Complex Changes:**
  - Scenario: If your project evolves, and significant changes to the database schema or data structure are needed.
  - Recommendation: Generate new migrations using the EF Core tools that reflect changes in your model. Use `dotnet ef migrations add <MigrationName>` to create migrations that properly capture the changes.

Run the project

Run the project with dotnet run or use the debugging mode, you should see the exact same results. But this time, the data access logic and business logic are separated. The DAL handles the data access logic, and the controllers handle the business logic. The repository pattern of OrderController and CustomerController are left for you to explore.

Service Injection and Lifetime

Service injection

The idea is to create some objects (named as services) to be used throughout the web app. Service injection (Dependency Injection) allows ASP.NET Core to handle creating, configuring, and supplying object instances throughout your app.

Why Register Services in Program.cs

Before ASP.NET Core can manage an object for you, you must register it in `Program.cs`. Registering a service adds it to ASP.NET Core's **Central Service Registry**. This acts as a central lookup directory that tells the framework three things:

- **Interface Mapping:** Which concrete class to instantiate when an interface is requested (e.g., map IItemRepository to ItemRepository).
- **Lifetime Management:** How long the object lives (Transient = new instance every time, Scoped = one instance per HTTP request, Singleton = one single instance for the app lifetime).
- **Nested Dependencies:** What nested dependencies the service itself needs so the framework can build the entire dependency tree automatically.

Why Dependency Injection (The Service Manager):

It is similar to when you go to a coffee shop, you simply tell the waiter (ASP.NET Core), *"Bring me a coffee"* (IItemRepository).

- In Program.cs, you already told the kitchen *how* to make coffee (ItemRepository).
- The waiter looks up the recipe, prepares the coffee behind the scenes, and delivers it straight to your table (the Controller). The controller does not need to create it again.

How Controllers Use It

When an HTTP request comes in, ASP.NET Core inspects the controller's constructor. It looks up what the controller needs in the **Service Registry**, creates the object automatically, and hands it over through **Constructor Injection**:

```
8   public class ItemController : Controller
9   {
        9 references
10      private readonly IItemRepository _itemRepository;
11
        0 references
12      public ItemController(IItemRepository itemRepository)
13      {
14          _itemRepository = itemRepository;
15      }
16
```

Service lifetime

Scoped:

A new instance of a scoped service is created for each http request.
The service instance is shared within the scope but not across different scopes.
This is a better option when you want to maintain a state within a request.

Transient:

A new instance of a transient service is created for each object in the HTTP request.

This is a good approach for the multithreading approach because both objects are independent of one another.

Singleton:
A singleton service provides a single instance throughout the lifetime of the application. Singleton services are suitable for components that need to maintain state or hold data that should be shared across the entire application.

To inject services with different lifetime, use the following code:

```
builder.Services.AddScoped<IService,Service>();
builder.Services.AddTransient<IService,Service>();
builder.Services.AddSingleton<IService,Service>();
```